# axodendron

## Deterministic neuronal morphology analysis and rendering from SWC

v0.1.0 2026-08-05 MIT

Analyze and visualize neuronal morphologies from SWC files.

Eito Yoneyama

`Axodendron` validates, analyzes, transforms, exports, and renders neuronal morphology data in Typst. A deterministic Rust/WebAssembly core owns SWC parsing, topology, morphometrics, transformations, and SVG geometry; Typst owns document layout and native annotations.

## Table of Contents

**Getting Started** . . . 3
I.1 Choosing a Validation Profile . . . 4
I.2 Cell Values . . . 4
**Compatibility and Architecture** . . . 5
**Example Data, Attribution, and License** . . . 6
**Validation and Provenance** . . . 8
IV.1 Profiles in Practice . . . 8
**Morphometric Analysis** . . . 9
V.1 Domains and Topology . . . 9
V.2 Sections, Paths, and Tortuosity . . . 10
V.3 Branch and Strahler Order . . . 10
V.4 Radius-Dependent Metrics . . . 11
**Sholl Analysis** . . . 12
**Pure Transformations** . . . 13
VII.1 Selection and Extraction . . . 13
VII.2 Rerooting and Pruning . . . 13
VII.3 Simplification . . . 14
VII.4 Resampling . . . 15
VII.5 Deterministic Export . . . 15
**Rendering** . . . 16
VIII.1 Projections . . . 16
VIII.2 Geometry and Radius Modes . . . 17
VIII.3 Soma Modes . . . 17
VIII.4 Type and Scalar Color . . . 17
VIII.5 Fitting and Aspect Ratio . . . 18
**Native Typst Annotations** . . . 19
IX.1 CeTZ Leader Labels . . . 20
IX.2 Render Reports . . . 21
**Publication Figure Guidance** . . . 23
**Error Model and Resource Limits** . . . 24
**API Reference** . . . 25
XII.1 Loading and Inspection . . . 25
XII.2 Analysis . . . 25
XII.3 Selection and Transformations . . . 26
XII.4 Annotation Builders . . . 27
XII.5 Renderer . . . 29
**Result Schemas** . . . 34
XIII.1 Analysis Bundle . . . 34
XIII.2 Transform Result . . . 34
XIII.3 Render Result . . . 34
**Limitations** . . . 35

**License, Dependencies, and Data Citations** ..... **36**
**Index** ..... **37**

# Part I

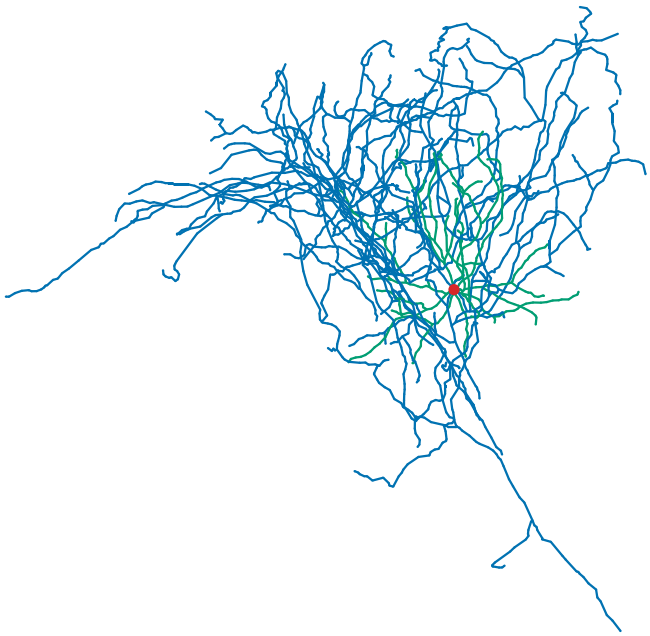
# Getting Started

Import `axodendron`, read an SWC file as bytes, validate it, and pass the resulting immutable cell to `#render`. The filename in a user document is resolved by the calling document, not by the package.

```
#import "@preview/axodendron:0.1.0" as swc

#let cell = swc.load(
    read("Sst-IRES-Cre_Ai14-188740-03-02-01_491119369_m.kp12.swc",  encoding:
none),
  profile: "incf-strict",
)

#swc.render(cell, width: 120mm, height: 90mm)
```



**Data:** `Sst-IRES-Cre_Ai14-188740-03-02-01_491119369_m.kp12.swc` NeuroMorpho.Org record 62495 Allen Cell Types doi:10.1016/j.neuron.2015.02.022 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

The examples below assume `#import "@preview/axodendron:0.1.0" as swc`. Code snippets treat the directory containing the selected `.swc` file as the caller root. This manual keeps its Typst sources and data separate, so its executable setup reads the same files from `examples/data/`.

On Typst 0.15.0 and later, `#load` can receive `path("AA0109.CNG.swc")` directly. Axodendron reads the path inside the package call while retaining caller-relative resolution. This manual uses

`read(..., encoding: none)` because the minimum supported compiler is Typst 0.14.0 and the documentation toolchain is pinned to Typst 0.14.2.

SWC coordinates and radii are plain numbers in `cell.units`, conventionally micrometres. Typst lengths such as `120mm`, `4pt`, and `8pt` control document layout only and are never mixed into morphology calculations.

## I.1 Choosing a Validation Profile

Use `profile: "incf-strict"` for publication inputs expected to satisfy INCF SWC 1.0 ordering. It requires sequential positive IDs, parent-before-child order, a first root with parent `-1`, and one connected root. Use `profile: "permissive"` when a structurally valid archive contains arbitrary positive IDs, out-of-order rows, or a rooted forest that must be preserved exactly.

> Neither profile repairs morphology. Duplicate IDs, missing parents, self-parenting, cycles, malformed rows, and non-finite geometry remain errors. Permissive mode relaxes ordering and connectedness only.

## I.2 Cell Values

A successful load returns a dictionary containing `valid`, `diagnostics`, `fingerprint`, `source-fingerprint`, `node-count`, `units`, `metadata`, and a private canonical `payload`. The semantic fingerprint ignores harmless text formatting but changes when morphology semantics change. The source fingerprint tracks the exact input bytes.

```
#let cell = swc.load(read("AA0109.CNG.swc", encoding: none))

#table(
  columns: 2,
  [Valid], [#cell.valid],
  [Nodes], [#cell.node-count],
  [Units], [#cell.units],
  [Semantic fingerprint], [#cell.fingerprint],
)
```

| | |
|---|---|
| **Valid** | true |
| **Nodes** | 2172 |
| **Units** | um |
| **Semantic fingerprint** | fnv1a64:b81103a39dfa984a |

**Data:** `AA0109.CNG.swc` NeuroMorpho.Org record 85226 MouseLight doi:10.1002/jnr.23978 and reconstruction deposit doi:10.25378/janelia.5526706 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

# Part II

# Compatibility and Architecture

| Surface | Minimum | Coverage |
|---|---|---|
| Typst package with string or bytes input | `0.14.0` | CI compiles the complete package and smoke suite on the minimum and latest configured Typst versions |
| Typst `path(...)` input | `0.15.0` | A version-gated smoke assertion loads a real file through `path(...)` |
| Rust workspace | `1.85.0` | Ubuntu minimum plus stable Rust on Ubuntu, macOS, and Windows |
| Detailed manual | `0.14.2` | Built reproducibly with `just docs` and the pinned Nixpkgs revision |

The package is split into a format-independent Rust core, an SVG renderer, and a thin Typst minimal-protocol adapter. The WASM plugin is pure and stateless: it cannot access files, clocks, randomness, environment variables, or the network. Equal request bytes produce equal response bytes.

The wire boundary uses deterministic CBOR with `api_version: 1`. Morphology payload schema 2 serializes canonical arrays, provenance, and fingerprints; derived children, roots, components, lookup maps, and soma classification are rebuilt and validated on every decode.

# Part III

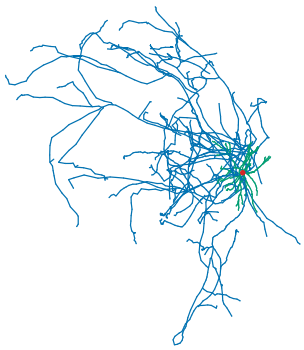
# Example Data, Attribution, and License

This manual uses four standardized real SWC reconstructions from NeuroMorpho.Org. They are distributed under CC BY 4.0[1]. Reuse requires attribution to the original study and NeuroMorpho.Org, RRID:SCR_002145. NeuroMorpho.Org also requests citation of its database publications, including Tecuatl, Ljungquist, and Ascoli (2024), doi:10.1096/fba.2024-00048[2].

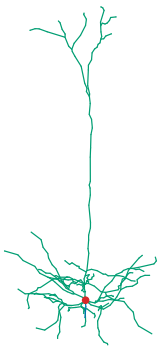
| File | Record | Archive | Original study |
|---|---|---|---|
| `AA0109.CNG.swc` | 85226[3] | MouseLight | 10.1002/jnr.23978[4] deposit[5] |
| `Nr5a1-Cre_Ai14-187777-05-02-01_491392821_m.kp1.swc` | 62390[6] | Allen Cell Types | 10.1016/j.neuron.2015.02.022[7] |
| `Sst-IRES-Cre_Ai14-188740-03-02-01_491119369_m.kp12.swc` | 62495[8] | Allen Cell Types | 10.1016/j.neuron.2015.02.022[9] |
| `Vipr2-IRES2-Cre_Ai14-310513-05-02-01_637021223_m.CNG.swc` | 102520[10] | Allen Cell Types | 10.1016/j.neuron.2015.02.022[11] |

```
#grid(
  columns: 2,
  gutter: 5mm,
  row-gutter: 5mm,
  swc.render(aa, width: 65mm, height: 48.75mm),
  swc.render(nr5a1, width: 65mm, height: 48.75mm),
  swc.render(sst, width: 65mm, height: 48.75mm),
  swc.render(vipr2, width: 65mm, height: 48.75mm),
)
```

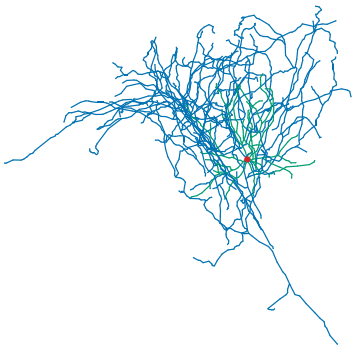
[1] https://creativecommons.org/licenses/by/4.0/
[2] https://doi.org/10.1096/fba.2024-00048
[3] https://neuromorpho.org/api/neuron/id/85226
[4] https://doi.org/10.1002/jnr.23978
[5] https://doi.org/10.25378/janelia.5526706
[6] https://neuromorpho.org/api/neuron/id/62390
[7] https://doi.org/10.1016/j.neuron.2015.02.022
[8] https://neuromorpho.org/api/neuron/id/62495
[9] https://doi.org/10.1016/j.neuron.2015.02.022
[10] https://neuromorpho.org/api/neuron/id/102520
[11] https://doi.org/10.1016/j.neuron.2015.02.022



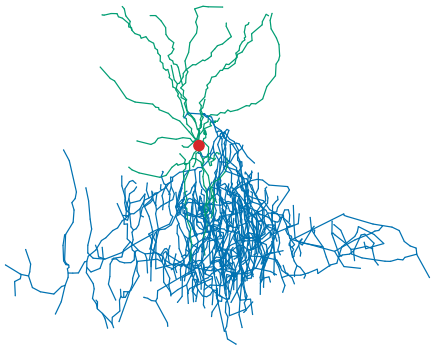
**Data:** `AA0109.CNG.swc` NeuroMorpho.Org record 85226 MouseLight doi:10.1002/jnr.23978 and reconstruction deposit doi:10.25378/janelia.5526706 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.



**Data:** `Nr5a1-Cre_Ai14-187777-05-02-01_491392821_m.kp1.swc` NeuroMorpho.Org record 62390 Allen Cell Types doi:10.1016/j.neuron.2015.02.022 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.



**Data:** `Sst-IRES-Cre_Ai14-188740-03-02-01_491119369_m.kp12.swc` NeuroMorpho.Org record 62495 Allen Cell Types doi:10.1016/j.neuron.2015.02.022 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.



**Data:** `Vipr2-IRES2-Cre_Ai14-310513-05-02-01_637021223_m.CNG.swc` NeuroMorpho.Org record 102520 Allen Cell Types doi:10.1016/j.neuron.2015.02.022 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

The real files are included only as attributed documentation and README examples. Their source URLs, SHA-256 checksums, archive names, original studies, and license are also recorded in `THIRD_PARTY_NOTICES.md`. The separate 30-case regression corpus remains an ignored, checksum-verified private cache and is not distributed.

# Part IV

# Validation and Provenance

`#diagnostics` returns every retained diagnostic as a dictionary with `code`, `severity`, `message`, optional `line`, optional `column`, and optional `node-id`. Set `⟨fail-on-error⟩` to `false` when a document should inspect invalid input instead of stopping immediately.

```
#let malformed = swc.from-text(
  "1 1 0 0 0 2 -1\n1 3 0 5 0 1 1\n",
  fail-on-error: false,
)

#for item in swc.diagnostics(malformed) [
  #item.severity: #item.code - #item.message
]
```

| Severity | Code | Message |
|---|---|---|
| error | SWC_DUPLICATE_ID | node id 1 duplicates line 1 |
| error | SWC_SELF_PARENT | node cannot be its own parent |
| warning | SWC_MULTIPLE_ROOTS | morphology has 2 roots; expected one |
| warning | SWC_DISCONNECTED_COMPONENT | morphology contains 2 disconnected components |

`#metadata` exposes recognized header fields without applying them. Scale, shrinkage correction, species, region, and free-form comments remain provenance. Axodendron never changes coordinates or radii because a header suggests a correction.

## IV.1 Profiles in Practice

`incf-strict` is appropriate for standardized NeuroMorpho.Org files and reproducible interchange. `permissive` is appropriate for structurally valid rooted forests or legacy tools that do not preserve row ordering. Exporting a single-root transformed cell through `#export-swc` produces sequential, topologically ordered IDs that can be loaded strictly again.

# Part V

# Morphometric Analysis

`#analyze` computes one versioned bundle containing a summary, topology, sections, tortuosity, and four node-aligned fields. The default `neurites` domain excludes type-1 soma nodes and every soma-incident edge; `raw` includes every encoded node and edge.

```
#let metrics = swc.analyze(cell)
#let summary = metrics.summary

#table(
  columns: 2,
  [Nodes], [#summary.node_count],
  [Cable length], [#summary.total_cable_length],
  [Branch points], [#summary.branch_point_count],
  [Terminals], [#summary.terminal_count],
)
```

| | |
|---|---|
| **Domain** | neurites |
| **Neurite nodes** | 2169 |
| **Cable length** | 90219.4 um |
| **Branch points** | 119 |
| **Terminals** | 127 |
| **Sections** | 245 |
| **Maximum root path** | 6554.31 um |

**Data:** `AA0109.CNG.swc` NeuroMorpho.Org record 85226 MouseLight doi:10.1002/jnr.23978 and reconstruction deposit doi:10.25378/janelia.5526706 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

## V.1 Domains and Topology

Each non-soma node whose parent is excluded becomes an independent arbor root in the `neurites` domain. A node with more than one included child is a branch point; a node with no included child is terminal. `topology.node-ids`, `parent-ids`, `root-ids`, `terminal-ids`, `branch-point-ids`, and `component-ids` are aligned and deterministic.

Use `domain: "raw"` only when a measurement should follow the encoded graph convention, including soma nodes and soma connectors. The distinction matters when comparing cable length or path distance with external tools.

## V.2 Sections, Paths, and Tortuosity

The default `section-boundaries: "topology-and-type"` starts or ends a section at roots, branch points, terminals, and SWC type changes. The transition edge belongs to the proximal section, so section lengths sum exactly to total cable length. `"topology-only"` ignores type changes.

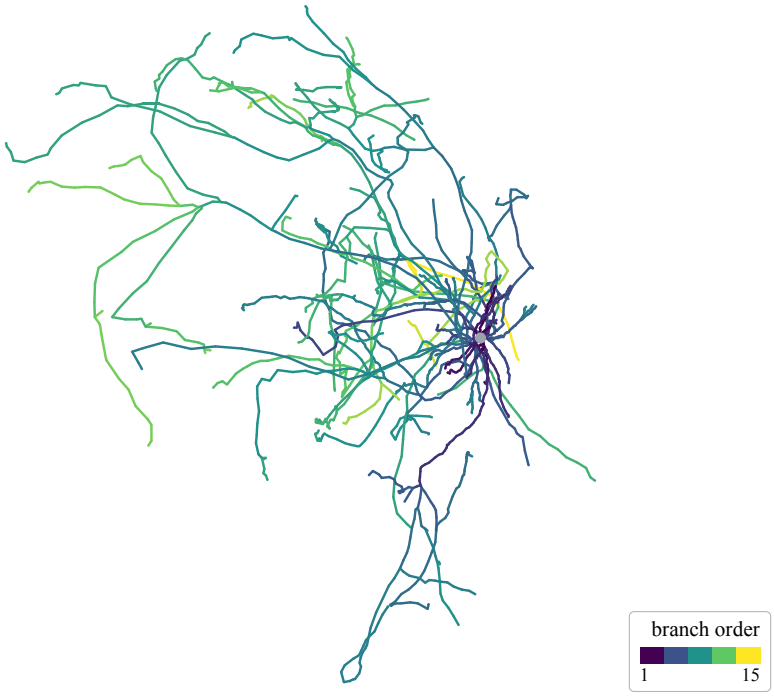
Root path length accumulates 3D Euclidean segment lengths from each arbor root. Radial distance is the straight-line 3D distance from that root. Section tortuosity is path length divided by endpoint distance; zero-span sections are excluded and counted separately.

## V.3 Branch and Strahler Order

Centrifugal branch order is `1` at each arbor root and increments on children of a branch point. Strahler order is computed independently per arbor: terminals are `1`, and a parent increments the maximum child order only when that maximum occurs at least twice.

```
#let metrics = swc.analyze(cell)

#swc.render(
  cell,
  color-by: metrics.branch_order,
  colormap: "viridis",
  minimum: 1,
  maximum: 20,
  color-bar: swc.color-bar(
    min: 1,
    max: 20,
    label: [branch order],
  ),
)
```




**Data:** `AA0109.CNG.swc` NeuroMorpho.Org record 85226 MouseLight doi:10.1002/jnr.23978 and reconstruction deposit doi:10.25378/janelia.5526706 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

Node fields include `name`, `node-ids`, `values`, `units`, `fingerprint`, `domain`, and `definition-version`. Passing a field from another cell to `#render` is rejected instead of silently assigning values to the wrong geometry.

## V.4 Radius-Dependent Metrics

Each neurite segment is modeled as an uncapped circular frustum with axial length $L$ and endpoint radii $r_0$, $r_1$:

$$\text{area} = \pi(r_0 + r_1)\sqrt{L^2 + (r_0 - r_1)^2}$$

$$\text{volume} = \pi L \frac{r_0^2 + r_0 r_1 + r_1^2}{3}$$

No end caps are added. A non-positive endpoint radius makes aggregate surface area and volume unavailable, and the offending node IDs are returned. Input radii are never repaired silently.

A single-point soma exposes sphere area and volume. A valid NeuroMorpho three-point soma exposes equivalent-sphere values and the encoded cylinder's lateral area and volume. Equal radii, endpoint distance, and endpoint opposition use a 1% scale-relative tolerance. Other soma encodings do not receive invented scalar metrics.

# Part VI

# Sholl Analysis

`#sholl` intersects original 3D segments with spheres. `#sholl-2d` first applies the chosen physical orthographic projection, then intersects the projected segments with circles. Display simplification is never used for either calculation.

```
#let result = swc.sholl(
  cell,
  radii: range(20, 220, step: 20),
)

#table(
  columns: 2,
  table.header([Radius], [Intersections]),
  ..result.bins.map(bin => (bin.radius, bin.intersections)).flatten(),
)
```

| Radius (um) | Intersections |
|---|---|
| 20 | 15 |
| 40 | 34 |
| 60 | 49 |
| 80 | 55 |
| 100 | 61 |
| 120 | 56 |
| 140 | 58 |
| 160 | 51 |
| 180 | 44 |
| 200 | 45 |

**Data:** `Sst-IRES-Cre_Ai14-188740-03-02-01_491119369_m.kp12.swc` NeuroMorpho.Org record 62495 Allen Cell Types doi:10.1016/j.neuron.2015.02.022 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

The default center is the represented soma center, or the sole root when no soma exists. Soma-free forests and disconnected soma subgraphs have no unique default; provide `⟨center⟩` or `⟨center-node⟩`. Intersections at segment parameter `t` in `(0, 1]` count, so a shared vertex is counted once and a tangency counts once.

# Part VII

# Pure Transformations

Every transformation returns a new cell. The input remains unchanged. Results include `transform-report`, `mapping`, and `lineage`; the report records source/result fingerprints, node counts, removed and inserted IDs, cable-length change, and any guaranteed deviation bound.

## VII.1 Selection and Extraction

`#select-nodes` and `#select-kinds` return the induced forest over retained original edges. They never invent an edge across a removed node. `#subtree` extracts a node and every descendant. `#path` extracts the unique undirected route between two nodes in one component.

```
#let selected = swc.select-kinds(cell, kinds: (3, 4))
#let branch = swc.subtree(cell, node: 12)
#let route = swc.path(cell, start: 4, end: 11)
```

| | |
|---|---|
| **Selected dendrites** | 477 nodes |
| **Subtree at node 12** | 42 nodes |
| **Path 4 to 11** | 8 nodes |

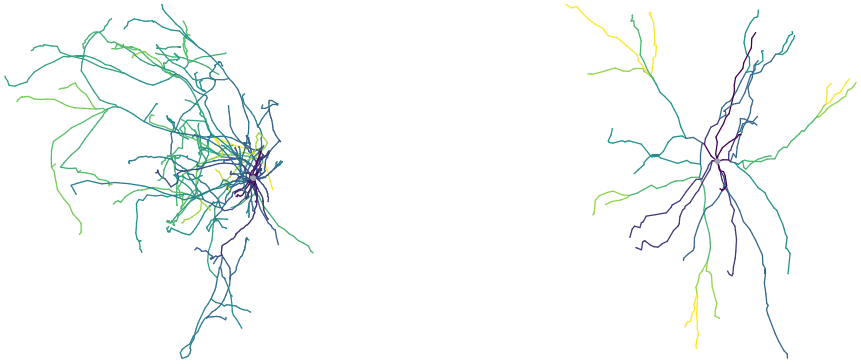
**Data:** `AA0109.CNG.swc` NeuroMorpho.Org record 85226 MouseLight doi:10.1002/jnr.23978 and reconstruction deposit doi:10.25378/janelia.5526706 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

## VII.2 Rerooting and Pruning

`#reroot` reverses the parent chain so the requested node becomes the root of its component. `#prune` removes every node of the listed kinds and each removed node's complete descendant subtree.

```
#let metrics = swc.analyze(cell)
#let dendrites = swc.prune(cell, kinds: (2,))
#let dendrite-metrics = swc.analyze(dendrites)

#grid(
  columns: 2,
  gutter: 5mm,
  swc.render(cell, color-by: metrics.branch_order),
  swc.render(dendrites, color-by: dendrite-metrics.branch_order),
)
```



**Data:** `AA0109.CNG.swc` NeuroMorpho.Org record 85226 MouseLight doi:10.1002/jnr.23978 and reconstruction deposit doi:10.25378/janelia.5526706 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

# VII.3 Simplification

`#simplify` applies topology-preserving 3D Ramer-Douglas-Peucker simplification independently between mandatory section points. Roots, branch points, terminals, protected IDs, soma nodes, and type changes are retained according to the options. The report's `guaranteed-max-deviation` is the requested tolerance.

Simplification is also available as the display-only `⟨display-tolerance⟩` option of `#render`. Display simplification never changes the source cell, analysis results, or export; requested native annotations, CeTZ labels, and explicit node anchors are protected automatically.

## VII.4 Resampling

`#resample` inserts equal-arc-length samples within non-soma, same-type topological sections. Roots, soma nodes, branch points, terminals, type boundaries, and protected IDs retain their original IDs. Radius interpolation is linear in arc length, and every inserted node receives proximal/distal lineage with a distal fraction.

```
#let sampled = swc.resample(cell, step: 5)

#table(
  columns: 2,
  [Source nodes], [#cell.node-count],
  [Result nodes], [#sampled.node-count],
  [Inserted nodes], [#sampled.lineage.len()],
)
```

| | |
|---|---|
| **Source nodes** | 2172 |
| **Result nodes** | 18175 |
| **Inserted nodes** | 17919 |
| **Cable-length change** | −296.80316729 |

**Data:** `AA0109.CNG.swc` NeuroMorpho.Org record 85226 MouseLight doi:10.1002/jnr.23978 and reconstruction deposit doi:10.25378/janelia.5526706 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

## VII.5 Deterministic Export

`#export-swc` emits canonical SWC with a deterministic header, topological row order, and sequential IDs. Single-root results satisfy `incf-strict`; forests retain every root and must be reloaded with `permissive` unless they are connected separately by the caller.

# Part VIII

# Rendering

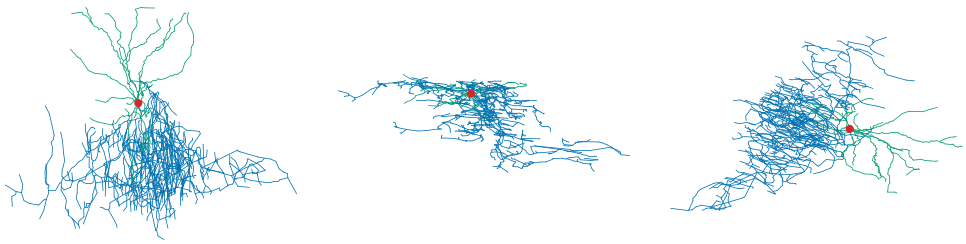
`#render` produces a compact deterministic SVG in WASM and places it in a Typst block. Labels, markers, legends, color bars, and scale bars are then composed as native Typst content.

## VIII.1 Projections

Named projections are `"xy"`, `"xz"`, and `"yz"`. An arbitrary orthographic camera is a dictionary containing `direction` and `up` vectors. The vectors must be finite, non-zero, and non-collinear.

```
#grid(
  columns: 3,
  gutter: 4mm,
  swc.render(cell, projection: "xy"),
  swc.render(cell, projection: "xz"),
  swc.render(
    cell,
    projection: (
      direction: (x: 1, y: 1, z: 1),
      up: (x: 0, y: 0, z: 1),
    ),
  ),
)
```

---



**Data:** `Vipr2-IRES2-Cre_Ai14-310513-05-02-01_637021223_m.CNG.swc` NeuroMorpho.Org record 102520 Allen Cell Types doi:10.1016/j.neuron.2015.02.022 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

## VIII.2 Geometry and Radius Modes

`geometry: "tapered"` draws radius-aware frusta with round distal joins. `"skeleton"` draws constant-width centerlines. `radius-mode: "physical"` maps projected radii linearly. `"readable"` applies ⟨`radius-exponent`⟩ and the screen-space ⟨`minimum-radius`⟩ and ⟨`maximum-radius`⟩ bounds.

```
#grid(
  columns: 2,
  gutter: 5mm,
  swc.render(cell, geometry: "tapered", radius-mode: "readable"),
  swc.render(cell, geometry: "skeleton"),
)
```

**Data:** `Nr5a1-Cre_Ai14-187777-05-02-01_491392821_m.kp1.swc` NeuroMorpho.Org record 62390 Allen Cell Types doi:10.1016/j.neuron.2015.02.022 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

## VIII.3 Soma Modes

`soma-mode: "equivalent-sphere"` is the default and displays one body at the projected soma centroid with the representative encoded radius. `"encoded"` uses cylinder geometry for a geometrically valid three-point soma. `"raw-points"` displays every encoded type-1 node and edge. Display policy does not invent soma area or volume metrics.

## VIII.4 Type and Scalar Color

The default `color-by: "type"` mapping uses red `#d62728` for soma, blue `#0072b2` for axon, and green `#009e73` for basal and apical dendrites. Types 5, 6, and 7 use `#e69f00`, `#56b4e9`, and `#cc79a7`; all other kinds use `#4d4d4d`.

Passing any other color string produces a uniform rendering. Passing a node field produces scalar color. Viridis is the default continuous map and Magma is also supported. Finite values are normalized linearly between ⟨`minimum`⟩ and ⟨`maximum`⟩, inferred from the field when omitted; values outside the range are clamped. Missing or non-finite values use neutral gray `#9ca3af`.

```
#let metrics = swc.analyze(cell)

#grid(
  columns: 2,
  gutter: 5mm,
  swc.render(cell, color-by: metrics.root_path_length),
  swc.render(
    cell,
    color-by: metrics.root_path_length,
    colormap: "magma",
  ),
)
```

**Data:** `AA0109.CNG.swc` NeuroMorpho.Org record 85226 MouseLight doi:10.1002/jnr.23978 and reconstruction deposit doi:10.25378/janelia.5526706 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

# VIII.5 Fitting and Aspect Ratio

Canvas fitting includes painted radii, outlines, and soma extent. It must not clip painted geometry inside `⟨padding⟩`. The Typst ratio `width / height` must exactly match `canvas-width / canvas-height`; Axodendron rejects a mismatch so overlays and physical scale bars remain calibrated.

`background: none` keeps the SVG transparent. Set `⟨outline-color⟩` to add a halo around segments and soma; `⟨outline-width⟩` has no effect when the color is `none`. Style strings accept a bounded safe subset of CSS color syntax and reject external URLs.

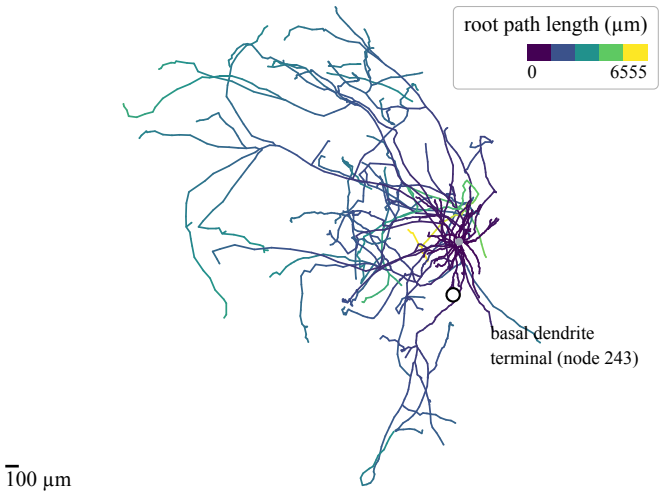
# Part IX

# Native Typst Annotations

`#label` and `#marker` anchor native Typst content to projected node IDs; `(offset)` then shifts it by `x` and `y` lengths without changing the selected node. `#legend`, `#color-bar`, and `#scale-bar` place publication-oriented overlays inside the render block. These overlays remain Typst content rather than being flattened into SVG text; `(label-gap)` controls the vertical separation between a color-bar label and its palette strip.

```
#let metrics = swc.analyze(cell)
#let path-maximum = calc.ceil(calc.max(..metrics.root_path_length.values))

#swc.render(
  cell,
  color-by: metrics.root_path_length,
  minimum: 0,
  maximum: path-maximum,
  labels: (swc.label(
    node: 243, offset: (x: 14pt, y: 12pt),
    text(size: 7pt)[basal dendrite #linebreak() terminal (node 243)],
  ),),
  markers: (swc.marker(node: 243),),
  color-bar: swc.color-bar(
    min: 0, max: path-maximum,
    label: [root path length (µm)], label-gap: 5pt, position: top + right,
  ),
  scale-bar: swc.scale-bar(value: 100),
)
```




**Data:** `AA0109.CNG.swc` NeuroMorpho.Org record 85226 MouseLight doi:10.1002/jnr.23978 and reconstruction deposit doi:10.25378/janelia.5526706 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

Unknown annotation IDs are errors by default. Set `⟨strict-node-ids⟩` to `false` only when optional annotations may legitimately target nodes removed by an earlier selection. Requested native labels, markers, CeTZ labels, and `⟨anchor-nodes⟩` are protected from display simplification.

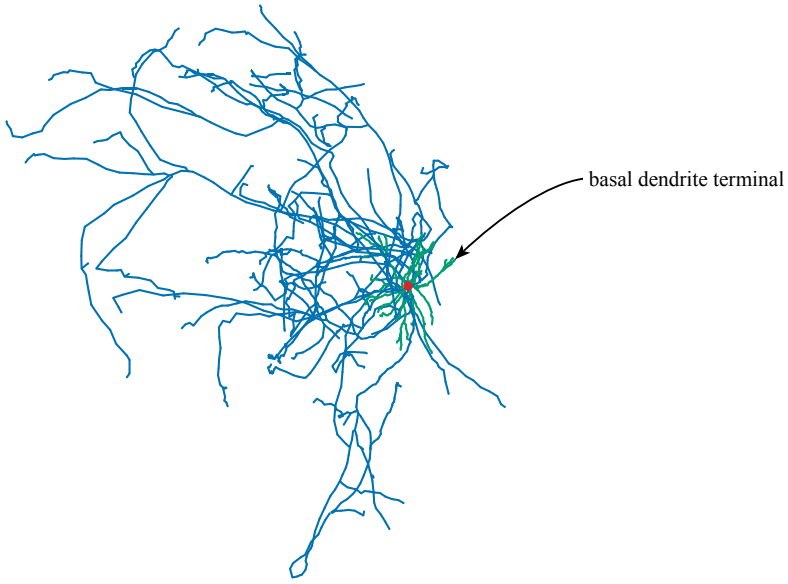
Label offsets and legend or color-bar positions are explicit layout choices rather than automatic collision-avoidance requests. Tune a node label with `offset: (x: ..., y: ...)`, choose a clear overlay corner with `⟨position⟩`, and use `⟨label-gap⟩` when the color-bar title needs more or less separation from the palette; this example labels SWC type-3 terminal node 243, marks the same node with a circle, uses a 14 pt rightward and 12 pt downward label shift, places the color bar in the clear top-right corner, and applies a 5 pt label gap.

## IX.1 CeTZ Leader Labels

CeTZ leader labels keep the text box away from morphology geometry while an arrow identifies the exact projected node. Import CeTZ separately and pass its module through `⟨cetz⟩` Axodendron has no mandatory CeTZ dependency and performs no second projection. `#render` requests and protects every `⟨cetz-labels⟩` target in the same WASM call, converts its final fitted screen coordinate to the CeTZ bottom-left coordinate system, registers it as a named CeTZ anchor, and composes the SVG, leader, and label in one canvas.

```
#import "@preview/axodendron:0.1.0" as swc
#import "@preview/cetz:0.5.2"

#swc.render(
  cell,
  width: 120mm,
  height: 90mm,
  cetz: cetz,
  cetz-labels: (swc.cetz-label(
    node: 447,
    offset: (x: 17mm, y: -9mm),
    controls: (
      (x: 12mm, y: -10mm),
      (x: 5mm, y: -5mm),
    ),
    text(size: 8pt)[basal dendrite terminal],
  ),),
)
```




**Data:** `AA0109.CNG.swc` NeuroMorpho.Org record 85226 MouseLight doi:10.1002/jnr.23978 and reconstruction deposit doi:10.25378/janelia.5526706 CC BY 4.0. Cite NeuroMorpho.Org, RRID:SCR_002145.

The `⟨offset⟩`, each `⟨via⟩` point, and each `⟨controls⟩` point are relative to the selected node in Typst screen coordinates: positive `x` moves right and positive `y` moves down. With `anchor: auto`, Axodendron places the label by the edge or corner facing the node. For a label beside the node, the leader itself starts at the `mid-west` or `mid-east` content anchor, so its initial `y` coordinate is the text's typographic vertical center rather than a corner. The default leader is straight; `⟨via⟩` inserts explicit straight segments. One `⟨controls⟩` point selects a quadratic CeTZ Bezier and two select a cubic Bezier. Axodendron never introduces curvature on its own. The default white fill and 2 pt padding keep morphology lines out of the text without a visible border. Set `⟨target-gap⟩` only when the arrow tip should stop short of the node.

A leader path is not an automatic obstacle-avoidance solver. A direct arrow can still cross an unrelated branch in a dense arbor, so use `⟨via⟩` for a deliberate polyline or `⟨controls⟩` for a deliberate curve and verify the final PDF. The two explicit control points in this example produce the displayed cubic Bezier. Node 447 is an encoded type-3 terminal with no child; the leader reaches that exact rendered sample without crossing the neighboring arbor.

For a custom CeTZ composition, request sparse coordinates with `anchor-nodes: (447, ...)` and `return-report: true`, retrieve a record with `#node-anchor`, or pass the result to `#cetz-annotate`. `x` and `y` in each returned record are lengths from the top-left of the base render block; `x-ratio` and `y-ratio` are normalized screen coordinates, and `screen-x` and `screen-y` retain the fitted SVG coordinates.

## IX.2 Render Reports

Set `⟨return-report⟩` to `true` to receive `body`, dimensions, `node-anchors`, `report`, `pixels-per-unit`, `source-node-count`, and `rendered-node-count`. The report records radius and soma

modes, counts of radii floored or capped, the simplified node count, and the protected overlay node count.

| Field | Interpretation |
|---|---|
| `pixels-per-unit` | Projected SVG pixels per morphology coordinate unit after fitting and padding |
| `source-node-count` | Canonical nodes in the input cell |
| `rendered-node-count` | Nodes retained for SVG geometry after display-only simplification |
| `node-anchors` | Sparse projected-node records requested by labels, markers, CeTZ labels, or `anchor-nodes`; all rendered nodes when `include-nodes` is true |
| `floored-radius-count` | Rendered radii raised to `minimum-radius` |
| `capped-radius-count` | Rendered radii limited by `maximum-radius` or `maximum-soma-radius` |
| `simplified-node-count` | Source nodes omitted only from display geometry |
| `overlay-node-count` | Unique valid node IDs protected for native labels, markers, CeTZ labels, or explicit anchors |

The report is disclosure metadata, not a transformed morphology. Analysis and export still use the input cell. For a reproducible publication figure, retain the projection, page and canvas dimensions, display tolerance, radius and soma modes, scalar range, palette, and the report alongside the input fingerprint.

# Part X

# Publication Figure Guidance

Use the type palette when compartment identity matters and a sequential map only for an ordered scalar. Do not use branch-order color without a color bar in a figure intended for quantitative interpretation. State the analysis domain and units in the caption.

Use `radius-mode: "physical"` only when literal projected thickness is legible at the final size. The default readable mode is more robust for overview figures, but its exponent and screen-space caps must be reported when thickness is interpreted.

Keep labels sparse and outside dense arbors. Add a light outline only when a morphology crosses a colored or photographic background. Verify the final PDF at publication size; deterministic SVG structure does not guarantee identical rasterization across font and PDF toolchains.

> Display simplification, readable radius scaling, equivalent-sphere soma display, and projection are presentation operations. They never change the cell or scientific analysis, but a publication caption should disclose them when they affect interpretation.

# Part XI

# Error Model and Resource Limits

Plugin errors are structured before the Typst wrapper turns them into readable panics. Stable error families are:

| Family | Meaning |
|---|---|
| `API_*` | Malformed CBOR or incompatible protocol |
| `PAYLOAD_*` | Incompatible, forged, or internally inconsistent morphology payload |
| `SWC_*` | Lexical, structural, or validation failure |
| `SHOLL_*` | Invalid center, radius, domain, or projection |
| `TRANSFORM_*` | Invalid node, component, tolerance, step, or ID space |
| `RENDER_*` | Invalid canvas, style, scalar field, projection, or overlay |
| `LIMIT_*` | Explicit bounded-resource failure |

| Resource | Limit |
|---|---|
| SWC source / encoded request | 64 MiB / 64 MiB |
| Morphology nodes | 250,000 |
| Decoded payload / encoded response | 128 MiB / 128 MiB |
| Sholl radii | 10,000 |
| SVG result | 64 MiB |
| WASM artifact | 1 MiB |
| Source package bundle | 4 MiB |

Limits are checked before unbounded output allocation. The normal quality gate also exercises the full 250,000-node parser boundary, bounded resampling expansion, a 100,000-node performance case, two clean byte-identical WASM builds, and package import compilation.

# Part XII

# API Reference

The public string `version` reports the embedded plugin version. The public `swc` dictionary mirrors every function below for selective dictionary imports, but dictionary-stored functions require parenthesized calls such as `(swc.analyze)(cell)`. Importing the package as a module and calling `swc.analyze(cell)` remains the recommended interface.

## XII.1 Loading and Inspection

**`#load((source), (profile): "permissive", (fail-on-error): true)` → `cell`**

Parse and validate SWC input.

Argument

`(source)` — `str` | `bytes` | `path`

SWC text or bytes. Typst 0.15.0 and later may pass `path(...)`; Typst 0.14.x should pass `read(..., encoding: none)` output.

Argument

`(profile): "permissive"` — `str`

`"permissive"` or `"incf-strict"`.

Argument

`(fail-on-error): true` — `bool`

Panic on validation errors when `true`; return an invalid diagnostic-bearing cell when `false`.

**`#from-text((source), (profile): "permissive", (fail-on-error): true)` → `cell`**

Parse SWC source already held in a string or bytes value. Arguments and validation behavior match `#load` except that `path` input is not read.

**`#diagnostics((cell))` → `array`**

Return the retained diagnostic dictionaries.

**`#metadata((cell))` → `dictionary`**

Return retained comments and recognized header fields without applying corrections.

## XII.2 Analysis

**`#analyze((cell), (domain): "neurites", (section-boundaries): "topology-and-type")` → `analysis`**

Compute the complete versioned morphometrics bundle.

Argument

`(domain): "neurites"` — `str`

`"neurites"` excludes soma nodes and soma-incident edges; `"raw"` includes the encoded graph.

Argument

`⟨section-boundaries⟩: "topology-and-type"` `str`

`"topology-and-type"` or `"topology-only"`.

```
#sholl(
  ⟨cell⟩,
  ⟨radii⟩: none,
  ⟨center⟩: none,
  ⟨center-node⟩: none,
  ⟨domain⟩: "neurites",
  ⟨projection⟩: none
) → dictionary
```

Compute exact 3D sphere intersections. Supplying `⟨projection⟩` switches to physical 2D projected intersections and is normally exposed through `#sholl-2d`.

```
#sholl-2d(
  ⟨cell⟩,
  ⟨radii⟩: none,
  ⟨projection⟩: "xy",
  ⟨center⟩: none,
  ⟨center-node⟩: none,
  ⟨domain⟩: "neurites"
) → dictionary
```

Compute exact 2D circle intersections after the requested orthographic projection.

## XII.3 Selection and Transformations

`#select-nodes(⟨cell⟩, ⟨nodes⟩: none)` → `cell`

Select exactly the listed node IDs as an induced forest.

`#select-kinds(⟨cell⟩, ⟨kinds⟩: none)` → `cell`

Select nodes with the listed SWC kinds as an induced forest.

`#subtree(⟨cell⟩, ⟨node⟩: none)` → `cell`

Extract a node and all descendants.

`#path(⟨cell⟩, ⟨start⟩: none, ⟨end⟩: none)` → `cell`

Extract the unique undirected path between two nodes in one component.

`#reroot(⟨cell⟩, ⟨node⟩: none)` → `cell`

Reverse a component's parent chain so `⟨node⟩` becomes its root.

`#prune(⟨cell⟩, ⟨kinds⟩: none)` → `cell`

Remove listed SWC kinds and their complete descendant subtrees.

```
#simplify(
  ⟨cell⟩,
  ⟨tolerance⟩: none,
  ⟨preserve-type-changes⟩: true,
  ⟨preserve-soma⟩: true,
  ⟨protected-nodes⟩: ()
) → cell
```

Apply topology-preserving 3D Ramer-Douglas-Peucker simplification.

`#resample(⟨cell⟩, ⟨step⟩: none, ⟨protected-nodes⟩: ()) → cell`

Resample eligible sections at equal arc-length spacing and return interpolation lineage.

`#export-swc(⟨cell⟩) → str`

Return deterministic canonical SWC text with sequential IDs and topological row order.

## XII.4 Annotation Builders

`#label(⟨body⟩, ⟨node⟩: none, ⟨offset⟩: (x: 4pt, y: -4pt)) → annotation`

Construct a Typst-native label anchored at a node. `⟨offset⟩` contains Typst `x` and `y` lengths applied after projection; positive `x` moves right and positive `y` moves down.

```
#marker(
  ⟨node⟩: none,
  ⟨body⟩: none,
  ⟨offset⟩: (x: 0pt, y: 0pt),
  ⟨size⟩: 5pt,
  ⟨fill⟩: luma(100%),
  ⟨stroke⟩: 0.8pt + luma(0%)
) → annotation
```

Construct a node marker. When `⟨body⟩` is omitted, a circle is drawn.

```
#cetz-label(
  ⟨body⟩,
  ⟨node⟩: none,
  ⟨offset⟩: (x: 45.35pt, y: -28.35pt),
  ⟨via⟩: (),
  ⟨controls⟩: (),
  ⟨anchor⟩: auto,
  ⟨padding⟩: 2pt,
  ⟨fill⟩: luma(100%),
  ⟨label-stroke⟩: none,
  ⟨arrow-stroke⟩: 0.7pt + luma(0%),
  ⟨arrow-fill⟩: luma(0%),
  ⟨mark⟩: "stealth",
  ⟨mark-scale⟩: 0.7,
  ⟨target-gap⟩: 0pt
) → annotation
```

Construct an optional CeTZ leader label. ⟨offset⟩, every ⟨via⟩ dictionary, and every ⟨controls⟩ dictionary contain Typst x and y lengths relative to the node; positive x moves right and positive y moves down. ⟨anchor⟩ accepts auto or a CeTZ content-anchor string for label placement. A side leader starts at the text's typographic vertical center. With no ⟨controls⟩ it is a straight line or a ⟨via⟩ polyline; one control selects a quadratic CeTZ Bezier and two select a cubic Bezier. ⟨controls⟩ and ⟨via⟩ cannot be combined. ⟨padding⟩ creates space around the text, ⟨fill⟩ and ⟨label-stroke⟩ style its rectangular background, and the remaining arguments style or shorten the leader and arrow tip.

**#node-anchor(⟨render-result⟩, ⟨node⟩: none)** → **dictionary**

Retrieve one projected-node record from a render result. Request the ID with ⟨anchor-nodes⟩, a native annotation, or a CeTZ label and set ⟨return-report⟩ to true.

```
#cetz-annotate(
  ⟨render-result⟩,
  ⟨cetz⟩: none,
  ⟨labels⟩: (),
  ⟨length⟩: 1pt,
  ⟨strict⟩: true
) → content
```

Compose values returned by #cetz-label over an existing render result. This two-stage form is useful when the same projected coordinates also drive caller-owned CeTZ elements; the one-stage ⟨cetz-labels⟩ renderer argument is preferred for ordinary leader labels.

**#legend(⟨entries⟩: none, ⟨position⟩: right + top, ⟨inset⟩: 8pt)** → **dictionary**

Construct a compact categorical legend. Each entry has label and color.

```
#color-bar(
  ⟨min⟩: none,
  ⟨max⟩: none,
  ⟨label⟩: none,
  ⟨label-gap⟩: 4pt,
  ⟨colormap⟩: "viridis",
  ⟨position⟩: right + bottom,
  ⟨inset⟩: 8pt
) → dictionary
```

Construct a scalar color bar using the renderer's named palette. ⟨min⟩ and ⟨max⟩ are aligned to the exact left and right ends of the palette strip. ⟨label-gap⟩ is the non-negative vertical space between ⟨label⟩ and the palette strip.

**#scale-bar(⟨value⟩: none, ⟨label⟩: none, ⟨inset⟩: 8pt, ⟨stroke⟩: 1pt)** → **dictionary**

Construct a physical scale bar. ⟨value⟩ is expressed in cell.units.

## XII.5 Renderer

```
#render(
  (cell),
  (projection): "xy",
  (color-by): "type",
  (colormap): "viridis",
  (minimum): none,
  (maximum): none,
  (width): 340.16pt,
  (height): 255.12pt,
  (geometry): "tapered",
  (radius-mode): "readable",
  (soma-mode): "equivalent-sphere",
  (canvas-width): 800,
  (canvas-height): 600,
  (padding): 24,
  (stroke-width): 2,
  (minimum-radius): 1,
  (maximum-radius): 18,
  (maximum-soma-radius): 96,
  (radius-scale): 1,
  (radius-exponent): 0.5,
  (soma-scale): 1,
  (background): none,
  (outline-color): none,
  (outline-width): 1,
  (display-tolerance): none,
  (include-nodes): false,
  (anchor-nodes): (),
  (labels): (),
  (markers): (),
  (cetz): none,
  (cetz-labels): (),
  (legend): none,
  (color-bar): none,
  (scale-bar): none,
  (strict-node-ids): true,
  (return-report): false
) → content
```

Render deterministic SVG and compose optional native Typst overlays.

Argument

(projection): "xy" — str | dictionary

"xy", "xz", "yz", or (direction:, up:).

Argument
⟨color-by⟩: "type" str | node-field
"type", a safe CSS color string, or a fingerprint-bound scalar node field.

Argument
⟨colormap⟩: "viridis" str
"viridis" or "magma" for scalar fields.

Argument
⟨minimum⟩: none float | int | none
Scalar range minimum; inferred from finite field values when omitted.

Argument
⟨maximum⟩: none float | int | none
Scalar range maximum; inferred from finite field values when omitted.

Argument
⟨width⟩: 340.16pt length
Typst output width. Its ratio with ⟨height⟩ must match the numeric canvas ratio.

Argument
⟨height⟩: 255.12pt length
Typst output height.

Argument
⟨geometry⟩: "tapered" str
"tapered" or "skeleton".

Argument
⟨radius-mode⟩: "readable" str
"readable" or "physical".

Argument
⟨soma-mode⟩: "equivalent-sphere" str
"equivalent-sphere", "encoded", or "raw-points".

Argument
⟨canvas-width⟩: 800 float | int
Numeric SVG width in pixels. Its ratio with ⟨canvas-height⟩ must equal the Typst output ratio.

Argument

`⟨canvas-height⟩: 600` `float` | `int`

Numeric SVG height in pixels.

Argument

`⟨padding⟩: 24` `float` | `int`

Minimum fitted clearance in SVG pixels, including painted radii, soma extent, and outlines.

Argument

`⟨stroke-width⟩: 2` `float` | `int`

Skeleton stroke width and tapered-geometry distal join allowance in SVG pixels.

Argument

`⟨minimum-radius⟩: 1` `float` | `int`

Screen-space lower bound for rendered radii in readable mode.

Argument

`⟨maximum-radius⟩: 18` `float` | `int`

Screen-space upper bound for non-soma radii in readable mode.

Argument

`⟨maximum-soma-radius⟩: 96` `float` | `int`

Independent screen-space upper bound for soma radii in readable mode.

Argument

`⟨radius-scale⟩: 1` `float` | `int`

Multiplicative scale applied to neurite radii before screen-space bounds.

Argument

`⟨radius-exponent⟩: 0.5` `float` | `int`

Readable-mode exponent applied to projected radii; `1` preserves linear scaling before bounds.

Argument

`⟨soma-scale⟩: 1` `float` | `int`

Multiplicative scale applied to the displayed soma radius.

Argument

`⟨background⟩: none` `str` | `none`

Safe CSS color string for the SVG canvas, or `none` for transparency.

Argument
`⟨outline-color⟩: none` `str` | `none`

Safe CSS color string for a segment and soma halo, or `none` to disable it.

Argument
`⟨outline-width⟩: 1` `float` | `int`

Outline width in SVG pixels; ignored when `⟨outline-color⟩` is `none`.

Argument
`⟨display-tolerance⟩: none` `float` | `int` | `none`

Optional morphology-unit tolerance for display-only topology-preserving simplification.

Argument
`⟨include-nodes⟩: false` `bool`

Render every retained sample node as an SVG marker and include every projected node in `⟨node-anchors⟩`. Leave this off for ordinary figures and request sparse coordinates with `⟨anchor-nodes⟩`.

Argument
`⟨anchor-nodes⟩: ()` `array`

Integer node IDs whose final projected coordinates should be protected from display simplification and included in the render result without drawing markers.

Argument
`⟨labels⟩: ()` `array`

Sequence returned by `#label`. Target IDs are protected from display simplification.

Argument
`⟨markers⟩: ()` `array`

Sequence returned by `#marker`. Target IDs are protected from display simplification.

Argument
`⟨cetz⟩: none` `function` | `none`

Caller-imported CeTZ module. Required only when `⟨cetz-labels⟩` is non-empty; Axodendron does not import CeTZ itself.

Argument
`⟨cetz-labels⟩: ()` `array`

Sequence returned by `#cetz-label`. Targets are protected, projected, registered as CeTZ anchors, and composed over the base render.

Argument

`⟨legend⟩: none` `dictionary` | `none`

Overlay returned by `#legend`.

Argument

`⟨color-bar⟩: none` `dictionary` | `none`

Overlay returned by `#color-bar`.

Argument

`⟨scale-bar⟩: none` `dictionary` | `none`

Overlay returned by `#scale-bar`.

Argument

`⟨strict-node-ids⟩: true` `bool`

Reject unknown native-label, marker, CeTZ-label, or explicit anchor IDs. If `false`, missing optional annotations are skipped.

Argument

`⟨return-report⟩: false` `bool`

Return a dictionary containing `body`, base dimensions, projected node anchors, and fit, simplification, radius, and overlay metadata instead of content alone.

# Part XIII
# Result Schemas

## XIII.1 Analysis Bundle

The analysis bundle contains `schema-version`, `definition-version`, `fingerprint`, `domain`, `summary`, `topology`, `sections`, `tortuosity`, `root-path-length`, `radial-distance`, `branch-order`, and `strahler-order`. The current definition version is `axodendron-morphometrics-1`.

The summary includes raw/domain node counts, edges, roots, components, branches, terminals, sections, total cable length, maximum path and radial distances, bounding box, type counts and metrics, per-arbor metrics, radius metrics, soma metrics and class, and units.

## XIII.2 Transform Result

Transformed cells add `transform-report`, `mapping`, and `lineage`. `mapping` contains `old-id` and optional `new-id`; `lineage` contains `new-id`, `proximal-old-id`, `distal-old-id`, and `distal-fraction` for inserted samples.

## XIII.3 Render Result

With `return-report: true`, the renderer returns `body`, `width`, `height`, `canvas-width`, `canvas-height`, `node-anchors`, `report`, `pixels-per-unit`, `source-node-count`, and `rendered-node-count`. A node-anchor record contains `node`, top-left-relative `x` and `y` lengths, normalized `x-ratio` and `y-ratio`, SVG `screen-x` and `screen-y`, and projected `depth`. Use `body` as content and retain the result when a reproducible figure must disclose display simplification, radius clamping, or annotation coordinates.

# Part XIV

# Limitations

- Axodendron renders orthographic views only; it does not provide perspective projection, lighting, or volumetric microscopy rendering.
- SWC expresses centerlines and sample radii, not membrane meshes, synapses, confidence intervals, or image registration. Axodendron does not infer information absent from the file.
- The default readable radius mode is a display policy, not a literal physical cross-section. Use physical mode and report it when width carries quantitative meaning.
- Equivalent-sphere soma display is a visual summary. Only supported soma encodings receive documented area and volume metrics.
- Scalar color is linear and supports Viridis or Magma. Axodendron does not automatically choose diverging, logarithmic, or categorical maps for a scientific hypothesis.
- Native and CeTZ labels use exact projected node anchors and explicit offsets; CeTZ `via` points define polylines and `controls` define Bezier curves, but collision-free placement is not automatic.
- Rendering tests guarantee deterministic SVG structure and broad real-data coverage, not identical raster pixels across PDF engines and fonts.

# Part XV

# License, Dependencies, and Data Citations

Axodendron source is distributed under the MIT license. The compiled WASM includes `ciborium`, `serde`, `half`, `cfg-if`, `zerocopy`, and `wasm-minimal-protocol`; their selected licenses and exact versions are recorded in `THIRD_PARTY_NOTICES.md` and `wasm-plugin/Cargo.lock`. CeTZ 0.5.2 is an optional LGPL-3.0-or-later example and documentation dependency supplied by the caller; its code is not embedded in Axodendron or `plugin.wasm`.

The four real SWC files used throughout this manual remain CC BY 4.0 material. Their per-file record links, archive names, original studies, and SHA-256 checksums are recorded in `THIRD_PARTY_NOTICES.md`. Every rendered occurrence in this manual also carries an adjacent attribution line.

When reusing a real-data figure, preserve its adjacent attribution, cite the original study, cite NeuroMorpho.Org with RRID:SCR_002145, and follow the NeuroMorpho.Org terms of use. The database citation used by this manual is Tecuatl C, Ljungquist B, Ascoli GA (2024), *Accelerating the continuous community sharing of digital neuromorphology data*, FASEB BioAdvances 6(7):207-221, doi:10.1096/fba.2024-00048[12].

> The detailed file-level notice is authoritative for bundled third-party material. This manual summarizes it for figure readers but does not replace `THIRD_PARTY_NOTICES.md`.

[12] https://doi.org/10.1096/fba.2024-00048

# Part XVI

# Index

## A


`analysis` ..... 25
#analyze ..... 25
`annotation` ..... 25


## C


`cell` ..... 25
#cetz-annotate ..... 28
#cetz-label ..... 27
#color-bar ..... 28


## D


#diagnostics ..... 25


## E


#export-swc ..... 27


## F


#from-text ..... 25


## L


#label ..... 27
#legend ..... 28
#load ..... 25


## M


#marker ..... 27
#metadata ..... 25


## N


#node-anchor ..... 28
`node-field` ..... 25


## P


#path ..... 26
#prune ..... 26


## R


#render ..... 29
`render-result` ..... 25
#reroot ..... 26
#resample ..... 27


## S


#scale-bar ..... 28
#select-kinds ..... 26
#select-nodes ..... 26
#sholl ..... 26
#sholl-2d ..... 26
#simplify ..... 27
#subtree ..... 26